8 Distributed tracing

This chapter covers

- What distributed tracing is and why it's important
- Seeing distributed tracing in action for troubleshooting
- Considerations on how and where to use distributed tracing
- How to go about selecting a distributed tracing solution

In the "good old days," when running a monolithic application, things were sort of convenient, from an observability perspective; you had a web server as the frontend tier, your application server (like a Java-based app), and a database such as MySQL. In this setup, it was not too hard to figure out which of the components caused an issue. The logs from these components would cover both the *where* and *what*. In addition, you may capture some metrics, indicating overall health and performance.

With the rise of cloud-native systems and microservices, from the end user perspective, nothing should change, or better put, they should not notice a difference in terms of the user experience, compared to a monolith. From a developer and operations perspective, we expect to benefit from microservices. This includes